

INFLUENCELIFT AI

Coding Guide

A 42-page implementation walkthrough for reading, explaining, testing, and extending the InfluenceLift AI codebase.

Portfolio Edition 1.0

Primary author	Mohit Bhatnagar
Collaboration credit	Aarushi Rai - marketing analytics framing and portfolio positioning
Live application	https://influencelift-ai.streamlit.app
Source repository	https://github.com/mohit231007/influencelift-ai

Purpose: help the project owner explain, defend, extend, and market the system in technical and business interviews.

How to use this guide

Follow the guide in repository order. Each page includes an implementation anchor that can be discussed in a coding or code-review interview.

This is a practical interview and portfolio manual. Each numbered module is intentionally self-contained so you can revise one topic, rehearse a two-minute answer, and then practise a deeper follow-up.

Module	Topic
01	Local environment and reproducible setup
02	Package layout and dependency management
03	Constants and feature schemas
04	Human-number parser implementation
05	Required-column validation
06	CleaningPolicy dataclass
07	Vectorised campaign cleaning
08	Row-level audit flags
09	Aggregate data-quality report
10	Scikit-learn custom transformer
11	Leakage-safe preprocessing pipeline
12	Ridge estimator configuration
13	Cross-validation evaluation function
14	Hyperparameter tuning pattern
15	Residual interval calculation
16	Training-range capture
17	Model bundle dataclass
18	Model serialization and safe loading
19	Deterministic demo-data generation
20	Command-line training script
21	Command-line batch prediction
22	Inference output assembly
23	Extrapolation warning logic
24	Scenario comparison function
25	FastAPI application factory
26	Pydantic request and response schemas
27	Single prediction endpoint
28	Batch prediction endpoint
29	Streamlit resource caching
30	Streamlit single-prediction form
31	Streamlit batch upload and download
32	Streamlit scenario simulator
33	Model information presentation
34	Exception handling strategy
35	Unit-testing parser and cleaning
36	Integration-testing training pipeline
37	API tests with TestClient
38	GitHub Actions CI matrix
39	Deployment smoke tests
40	Docker, QA script, and extension checklist

01. Local environment and reproducible setup

Objective: Understand and defend the implementation choices behind local environment and reproducible setup.

Core explanation

- Keep local environment and reproducible setup small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of local environment and reproducible setup follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -e ".[dev]"
ruff check .
pytest -q
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for local environment and reproducible setup.

02. Package layout and dependency management

Objective: Understand and defend the implementation choices behind package layout and dependency management.

Core explanation

- Keep package layout and dependency management small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of package layout and dependency management follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
[project]
name = "influencelift-ai"
requires-python = ">=3.10"
dependencies = ["pandas>=2.1,<3", "scikit-learn>=1.4,<2"]
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for package layout and dependency management.

03. Constants and feature schemas

Objective: Understand and defend the implementation choices behind constants and feature schemas.

Core explanation

- Keep constants and feature schemas small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of constants and feature schemas follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
RAW_FEATURE_COLUMNS = [
    "Followers", "EngagementRate (%)",
    "AdSpend (GBP)", "ContentQuality",
]
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for constants and feature schemas.

04. Human-number parser implementation

Objective: Understand and defend the implementation choices behind human-number parser implementation.

Core explanation

- Keep human-number parser implementation small, deterministic, typed, and independently testable.
- Place reusable behaviour in ``src/influencelift/``; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of human-number parser implementation follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
def parse_human_number(value):
    text = normalize(value)
    text = re.sub(r"[\$€£¥%s,]", "", text)
    match = FULL_NUMBER_PATTERN.fullmatch(text)
    return np.nan if not match else scale(match)
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for human-number parser implementation.

05. Required-column validation

Objective: Understand and defend the implementation choices behind required-column validation.

Core explanation

- Keep required-column validation small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of required-column validation follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
missing = [c for c in RAW_FEATURE_COLUMNS if c not in frame.columns]
if missing:
    raise ValueError(f"Missing required columns: {missing}")
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for required-column validation.

06. CleaningPolicy dataclass

Objective: Understand and defend the implementation choices behind cleaningpolicy dataclass.

Core explanation

- Keep cleaningpolicy dataclass small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of cleaningpolicy dataclass follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
@dataclass(frozen=True)
class CleaningPolicy:
    infer_small_followers_as_thousands: bool = True
    content_quality_min: float = 1.0
    content_quality_max: float = 10.0
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for cleaningpolicy dataclass.

07. Vectorised campaign cleaning

Objective: Understand and defend the implementation choices behind vectorised campaign cleaning.

Core explanation

- Keep vectorised campaign cleaning small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of vectorised campaign cleaning follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
followers = parse_series(frame["Followers"])
followers = followers.mask(followers.abs() < 1_000, followers * 1_000)
spend = parse_series(frame["AdSpend (GBP)"]).abs()
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for vectorised campaign cleaning.

08. Row-level audit flags

Objective: Understand and defend the implementation choices behind row-level audit flags.

Core explanation

- Keep row-level audit flags small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of row-level audit flags follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
flags = audit_row_corrections(frame)
# e.g. followers_explicit_scale_parsed, spend_currency_symbol_removed
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for row-level audit flags.

09. Aggregate data-quality report

Objective: Understand and defend the implementation choices behind aggregate data-quality report.

Core explanation

- Keep aggregate data-quality report small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of aggregate data-quality report follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
report = DataQualityReport(  
    rows=len(frame),  
    missing_by_column=missing,  
    correction_counts=corrections,  
    invalid_counts=invalid,  
    status=status,  
)
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for aggregate data-quality report.

10. Scikit-learn custom transformer

Objective: Understand and defend the implementation choices behind scikit-learn custom transformer.

Core explanation

- Keep scikit-learn custom transformer small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of scikit-learn custom transformer follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
class CampaignCleaner(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        validate_required_columns(X)
        return self
    def transform(self, X):
        return clean_campaign_frame(X, self.policy)
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for scikit-learn custom transformer.

11. Leakage-safe preprocessing pipeline

Objective: Understand and defend the implementation choices behind leakage-safe preprocessing pipeline.

Core explanation

- Keep leakage-safe preprocessing pipeline small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of leakage-safe preprocessing pipeline follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
Pipeline([
    ("clean", CampaignCleaner()),
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
    ("model", Ridge(alpha=1.0)),
])
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for leakage-safe preprocessing pipeline.

12. Ridge estimator configuration

Objective: Understand and defend the implementation choices behind ridge estimator configuration.

Core explanation

- Keep ridge estimator configuration small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of ridge estimator configuration follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
model = Ridge(alpha=alpha, random_state=None) # deterministic closed-form estimator
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for ridge estimator configuration.

13. Cross-validation evaluation function

Objective: Understand and defend the implementation choices behind cross-validation evaluation function.

Core explanation

- Keep cross-validation evaluation function small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of cross-validation evaluation function follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
scores = cross_validate(
    pipeline, X, y, cv=KFold(5, shuffle=True, random_state=42),
    scoring={"rmse": "neg_root_mean_squared_error", "r2": "r2"},
)
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for cross-validation evaluation function.

14. Hyperparameter tuning pattern

Objective: Understand and defend the implementation choices behind hyperparameter tuning pattern.

Core explanation

- Keep hyperparameter tuning pattern small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of hyperparameter tuning pattern follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
search = GridSearchCV(
    estimator=pipeline,
    param_grid={"model__alpha": [0.01, 0.1, 1, 10, 100]},
    scoring="neg_root_mean_squared_error", cv=5,
)
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for hyperparameter tuning pattern.

15. Residual interval calculation

Objective: Understand and defend the implementation choices behind residual interval calculation.

Core explanation

- Keep residual interval calculation small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of residual interval calculation follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
oof = cross_val_predict(pipeline, X, y, cv=cv)
residual_band = np.quantile(np.abs(y - oof), 0.95)
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for residual interval calculation.

16. Training-range capture

Objective: Understand and defend the implementation choices behind training-range capture.

Core explanation

- Keep training-range capture small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of training-range capture follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
ranges = {c: {"min": float(clean[c].min()), "max": float(clean[c].max())} for c in clean}
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for training-range capture.

17. Model bundle dataclass

Objective: Understand and defend the implementation choices behind model bundle dataclass.

Core explanation

- Keep model bundle dataclass small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of model bundle dataclass follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
@dataclass
class ModelBundle:
    pipeline: Pipeline
    model_version: str
    metrics: dict
    residual_quantile_95: float
    training_ranges: dict
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for model bundle dataclass.

18. Model serialization and safe loading

Objective: Understand and defend the implementation choices behind model serialization and safe loading.

Core explanation

- Keep model serialization and safe loading small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of model serialization and safe loading follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
joblib.dump(bundle, model_path)
# Load only artefacts produced by a trusted training pipeline.
bundle = joblib.load(model_path)
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for model serialization and safe loading.

19. Deterministic demo-data generation

Objective: Understand and defend the implementation choices behind deterministic demo-data generation.

Core explanation

- Keep deterministic demo-data generation small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of deterministic demo-data generation follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
rng = np.random.default_rng(seed)
followers = rng.integers(8_000, 210_000, rows)
noise = rng.normal(0, 2_000, rows)
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for deterministic demo-data generation.

20. Command-line training script

Objective: Understand and defend the implementation choices behind command-line training script.

Core explanation

- Keep command-line training script small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of command-line training script follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
python scripts/train.py --input data/generated/demo_train.csv --model-output artifacts/model_bundle.joblib
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for command-line training script.

21. Command-line batch prediction

Objective: Understand and defend the implementation choices behind command-line batch prediction.

Core explanation

- Keep command-line batch prediction small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of command-line batch prediction follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
python scripts/predict.py --input data/sample/synthetic_campaigns.csv --model artifacts/model_bundle.joblib --output artifacts/predictions.csv
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for command-line batch prediction.

22. Inference output assembly

Objective: Understand and defend the implementation choices behind inference output assembly.

Core explanation

- Keep inference output assembly small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of inference output assembly follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
prediction = np.maximum(bundle.pipeline.predict(frame), 0)
lower = np.maximum(prediction - band, 0)
upper = prediction + band
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for inference output assembly.

23. Extrapolation warning logic

Objective: Understand and defend the implementation choices behind extrapolation warning logic.

Core explanation

- Keep extrapolation warning logic small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of extrapolation warning logic follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
for feature, value in cleaned.iloc[0].items():
    if value < ranges[feature]["min"] or value > ranges[feature]["max"]:
        warnings.append(feature)
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for extrapolation warning logic.

24. Scenario comparison function

Objective: Understand and defend the implementation choices behind scenario comparison function.

Core explanation

- Keep scenario comparison function small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of scenario comparison function follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
uplift = proposed_sales - baseline_sales
cost_per_sale = spend_delta / uplift if uplift > 0 else None
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for scenario comparison function.

25. FastAPI application factory

Objective: Understand and defend the implementation choices behind fastapi application factory.

Core explanation

- Keep fastapi application factory small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of fastapi application factory follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
app = FastAPI(title="InfluenceLift AI API", version="1.0.0")
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for fastapi application factory.

26. Pydantic request and response schemas

Objective: Understand and defend the implementation choices behind pydantic request and response schemas.

Core explanation

- Keep pydantic request and response schemas small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of pydantic request and response schemas follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
class CampaignRequest(BaseModel):
    followers: str | float
    engagement_rate: str | float
    ad_spend: str | float
    content_quality: float
    timestamp: str | None = None
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for pydantic request and response schemas.

27. Single prediction endpoint

Objective: Understand and defend the implementation choices behind single prediction endpoint.

Core explanation

- Keep single prediction endpoint small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of single prediction endpoint follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
@app.post("/predict", response_model=PredictionResponse)
def predict(payload: CampaignRequest):
    frame = payload.to_frame()
    return predict_one(bundle, frame)
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for single prediction endpoint.

28. Batch prediction endpoint

Objective: Understand and defend the implementation choices behind batch prediction endpoint.

Core explanation

- Keep batch prediction endpoint small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of batch prediction endpoint follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
@app.post("/predict-batch")
def predict_batch(payload: BatchPredictionRequest):
    frame = pd.DataFrame([r.model_dump() for r in payload.records])
    return {"predictions": predict_campaigns(bundle, frame).to_dict("records")}
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for batch prediction endpoint.

29. Streamlit resource caching

Objective: Understand and defend the implementation choices behind streamlit resource caching.

Core explanation

- Keep streamlit resource caching small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of streamlit resource caching follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
@st.cache_resource
def model_bundle():
    return load_or_create_bundle()
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for streamlit resource caching.

30. Streamlit single-prediction form

Objective: Understand and defend the implementation choices behind streamlit single-prediction form.

Core explanation

- Keep streamlit single-prediction form small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of streamlit single-prediction form follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
followers = st.text_input("Followers", value="125K")
if st.button("Predict sales"):
    result = predict_campaigns(bundle, record).iloc[0]
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for streamlit single-prediction form.

31. Streamlit batch upload and download

Objective: Understand and defend the implementation choices behind streamlit batch upload and download.

Core explanation

- Keep streamlit batch upload and download small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of streamlit batch upload and download follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
uploaded = st.file_uploader("CSV file", type=["csv"])
predictions = predict_campaigns(bundle, pd.read_csv(uploaded))
st.download_button("Download predictions", predictions.to_csv(index=False))
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for streamlit batch upload and download.

32. Streamlit scenario simulator

Objective: Understand and defend the implementation choices behind streamlit scenario simulator.

Core explanation

- Keep streamlit scenario simulator small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of streamlit scenario simulator follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
comparison = compare_scenarios(bundle, baseline, proposed)
st.metric("Expected uplift", f"{comparison['expected_uplift_units']:+.}")
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for streamlit scenario simulator.

33. Model information presentation

Objective: Understand and defend the implementation choices behind model information presentation.

Core explanation

- Keep model information presentation small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of model information presentation follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
st.dataframe(pd.DataFrame([bundle.metrics]))
st.json(bundle.training_ranges)
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for model information presentation.

34. Exception handling strategy

Objective: Understand and defend the implementation choices behind exception handling strategy.

Core explanation

- Keep exception handling strategy small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of exception handling strategy follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
try:
    predictions = predict_campaigns(bundle, uploaded_frame)
except (TypeError, ValueError) as exc:
    st.error(str(exc))
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for exception handling strategy.

35. Unit-testing parser and cleaning

Objective: Understand and defend the implementation choices behind unit-testing parser and cleaning.

Core explanation

- Keep unit-testing parser and cleaning small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of unit-testing parser and cleaning follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
def test_parse_scaled_followers():
    assert parse_human_number("125K") == 125_000
    assert np.isnan(parse_human_number("unknown"))
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for unit-testing parser and cleaning.

36. Integration-testing training pipeline

Objective: Understand and defend the implementation choices behind integration-testing training pipeline.

Core explanation

- Keep integration-testing training pipeline small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of integration-testing training pipeline follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
def test_pipeline_trains_and_predicts(sample_frame, target):
    bundle = train_bundle(sample_frame, target)
    assert len(predict_campaigns(bundle, sample_frame)) == len(sample_frame)
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for integration-testing training pipeline.

37. API tests with TestClient

Objective: Understand and defend the implementation choices behind api tests with testclient.

Core explanation

- Keep api tests with testclient small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of api tests with testclient follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
client = TestClient(app)
response = client.post("/predict", json=payload)
assert response.status_code == 200
assert response.json()["predicted_sales_units"] >= 0
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for api tests with testclient.

38. GitHub Actions CI matrix

Objective: Understand and defend the implementation choices behind github actions ci matrix.

Core explanation

- Keep github actions ci matrix small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of github actions ci matrix follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
strategy:
  matrix:
    python-version: ["3.10", "3.11", "3.12"]
  steps:
    - run: pip install -e ".[dev]"
    - run: ruff check .
    - run: pytest -q
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for github actions ci matrix.

39. Deployment smoke tests

Objective: Understand and defend the implementation choices behind deployment smoke tests.

Core explanation

- Keep deployment smoke tests small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of deployment smoke tests follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
streamlit run app/Home.py --server.headless true &
curl --fail http://localhost:8501/_stcore/health
uvicorn api.main:app --port 8000 &
curl --fail http://localhost:8000/health
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for deployment smoke tests.

40. Docker, QA script, and extension checklist

Objective: Understand and defend the implementation choices behind docker, qa script, and extension checklist.

Core explanation

- Keep docker, qa script, and extension checklist small, deterministic, typed, and independently testable.
- Place reusable behaviour in `src/influencelift/`; keep Streamlit, FastAPI, and scripts as thin adapters.
- Preserve one transformation path for training and inference to avoid silent drift.
- Return explicit metadata and errors instead of relying on console inspection.

Portfolio defense answer

The implementation of docker, qa script, and extension checklist follows the repository's central rule: domain logic belongs in the package and interfaces should orchestrate rather than duplicate it. I prefer readable functions, dataclasses for configuration, pandas/scikit-learn compatibility, and tests that describe the expected business behaviour.

Evidence in InfluenceLift AI

- The project is packaged through `pyproject.toml` and imports from `src/influencelift`.
- Cleaning is represented as a scikit-learn transformer, so it is fitted and executed inside the pipeline.
- Automated lint, test, compile, and runtime smoke checks protect changes.

Implementation anchor

```
docker compose up --build
# Windows full QA
.\scripts\qa_local.ps1 -LaunchApp -LaunchApi
```

Trade-off to acknowledge

The code optimises clarity and portfolio inspectability over framework abstraction. If the package grows, interfaces, protocols, and dependency injection can be introduced where multiple implementations genuinely exist.

Likely follow-up: How would you test this behaviour and make it safe under production load?

Practice prompt: Open the relevant module, explain every public function, then propose one test and one safe refactor for docker, qa script, and extension checklist.